0 5 1

fixtures/verdict_cache/ — T self-validation fixtures

Fixture convention for `constitution/scripts/fastcycle/gates/verdict_cache.py` (contract `specs/004-fast-dev-cycles/contracts/affected-set-and-verdict-cache.md`, clauses VC-001..VC-005; data-model.md §5 "Verdict Cache Entry"; plan.md T-C01; research.md DEC-07). Task T051 (`tasks.md`) writes ONLY this RED test + these fixtures; the tool itself is T068, a later, separate task (Producer != Verifier, §11.4.240 — this file's author never implements `verdict_cache.py`).

Shared content (referenced by every scenario below)

File	Role
`gate_script.sh`	the fixture gate's script; its **bytes** are DEC-07 key component 1
`evidence.txt`	the fixture evidence artefact a `put --evidence <path>` records
`inputs/a.txt`	observed input #1, content `alpha-observed-input-000000000\n`
`inputs/a_flipped.txt`	`inputs/a.txt` with **exactly one byte** changed (`0` → `1` in the last digit; both files are 31 bytes, `cmp -l` confirms a single differing byte at offset 30) — stands in for "the real file at `inputs/a.txt` was re-traced after a one-byte edit"
`inputs/b.txt`	observed input #2, content `bravo-observed-input-000000000\n`
`inputs/c_irrelevant.txt`	a file the fixture gate never reads (never listed in any `observed_inputs`)
`inputs/c_irrelevant_changed.txt`	different content standing in for " `c_irrelevant.txt` changed for real on disk between put and get" — its own sha256 never appears in any envelope; it exists only so the negative-control scenario's claim "this file genuinely changed" is a measured fact, not an assertion

Every sha256 embedded in the envelope JSON files below is the REAL digest of the referenced `inputs/*` file (verified independently by both `sha256sum` and Python's `hashlib.sha256` at fixture-authoring time — the two agreed byte-for-byte; `test_verdict_cache_red.sh` re-verifies this agreement live on every run, §11.4.273 control needle, so a future hand-edit of `inputs/*.txt` without updating the matching envelope is caught rather than silently poisoning every fixture built on top of it).

`--inputs <trace.json>` envelope schema (UNCONFIRMED, binding on T)

The contract's own CLI line (`get|put|purge --cache-dir <dir> --gate <gate_id> [--inputs <trace.json>] [--verdict PASS|FAIL --evidence <path>]`) names 5 flags, but DEC-07/VC-001's key has 7 components (gate-script bytes, gate id, observed inputs, tool versions, env allow-list, target fingerprint, mutation id) and the contract does not pin how the last 4 reach the tool. This fixture set — like the established precedent in `fixtures/dispatch_stamp/` and `test_build_deploy_qa_events_red.sh` 's own "Contract for T040 implementer (UNCONFIRMED...)" section — DEFINES one concrete, defensible shape and flags it explicitly as binding-if-adopted, never silently assumed: `--inputs <trace.json>` is the FULL DEC-07 "observed context" envelope a `io_trace.sh trace` run (T-C02, a separate task) would emit for one gate, carrying every non-gate-script-byte, non-gate-id key component in one document:

```
{
  "gate_id": "GATE-VC-DEMO",
  "gate_script": "../gate_script.sh",
  "observed_inputs": [
    {"path": "inputs/a.txt", "sha256": "<hex>", "recorded_mtime": "<ISO-8601, NEVER part of the
            key>"},
    {"path": "inputs/b.txt", "sha256": "<hex>", "recorded_mtime": "<ISO-8601, NEVER part of the key>"}
  ],
  "tool_versions": {"python3": "3.11.4", "bash": "5.2.15"},
  "env_allowlist": {"LANG": "C.UTF-8", "TZ": "UTC"},
  "target_fingerprint": "hermetic",
  "mutation_id": "none"
}
```

`put` 's envelope additionally carries `"verdict"` (`PASS` / `FAIL` /other) — the value `--verdict` on the command line is expected to supply; it is committed here as `put_envelope.json` 's own field purely so one file documents the full `put` call, never as a claim that `get` 's envelope needs it (it does not — `get_envelope.json` omits `verdict` / `evidence` throughout, matching `get` 's own flag set). `recorded_mtime` is carried on each observed-input entry because a real tracer naturally records file metadata alongside content, but VC-002's own words ("content, not mtime, decides") and data-model §5's `recorded_at` field

("provenance only — not part of the key, so replays are byte-identical") both REQUIRE the key builder to ignore it; `vc_negctrl_mtime_only/` below is the fixture that proves this.

T068's implementer MAY choose a different envelope shape or pass some components as separate CLI flags instead — if so, per the same precedent, that choice belongs in `verdict_cache.py`'s own docstring, and this RED test's fixtures should then be read as "the semantic scenario", with the concrete envelope files adapted (never as an unreviewable, silent contract override).

Scenarios

Directory	Contract row / clause	Kind	Assertion
`unchanged_hit/`	tasks.md T051 "golden: unchanged inputs → hit"	golden-good	identical put/get envelopes ⇒ HIT
`vc_one_byte_flip/`	contract RED fixtures table row `vc_one_byte_flip`; FR-007	golden-bad	one observed input's recorded sha256 differs between put and get ⇒ MISS; the stale PASS is absent from `get`'s output
`vc_key_completeness_drop/`	tasks.md T051 "golden-bad: a key omitting one traced input is refused by the key-completeness check"	golden-bad	`get`'s `observed_inputs` list is missing one entry `put`'s list had ⇒ MISS
`vc_negctrl_mtime_only/`	tasks.md T051 "negative control: mtime-only change → hit"; plan.md T-C01 Protecting-tests line	negative-control	every `recorded_mtime` changes, every `sha256` stays identical ⇒ HIT
`vc_flip_tool_version/`	contract RED fixtures table row `vc_flip_tool_version`; plan.md T-C01 paired-mutation target ("drop the tool-version component from the key")	golden-bad	`tool_versions.python3` differs ⇒ MISS
`vc_flip_env/`	contract RED fixtures table row `vc_flip_env`	golden-bad	`env_allowlist.TZ` differs ⇒ MISS

Directory	Contract row / clause	Kind	Assertion
`vc_flip_mutation_id/`	contract RED fixtures table row `vc_flip_mutation_id`	golden-bad	`mutation_id` differs (`none` vs a real mutation run) ⇒ MISS — a paired-mutation verdict must never be served as the gate's real verdict, or vice versa
`vc_negctrl_irrelevant_file/`	contract RED fixtures table row `vc_negctrl_irrelevant_file`	negative-control	`inputs/c_irrelevant.txt` genuinely changes on disk but is never in `observed_inputs` ⇒ HIT (proves the cache is not vacuously always-miss)
`vc_admission_skip_never_cached/`	VC-003 admission rule ("Only PASS/FAIL ... are cached ... SKIP ... never cached")	golden-bad	`put --verdict SKIP` followed by an otherwise-identical `get` ⇒ MISS

Each scenario directory holds `put_envelope.json`, `get_envelope.json`, and `expected` (the literal `HIT` or `MISS`, plus for `vc_one_byte_flip` an extra non-blank line `STALE_PASS_ABSENT` the test greps for in `get`'s stdout — i.e. the stale verdict string must NOT appear).

What this RED test does NOT cover (honest gap, not silently assumed tested)

- **VC-003's "≥2 consecutive identical verdicts before admission"** rule needs a 3-call `put/put/put` sequence with a deliberately-differing middle verdict, not a single put/get pair; no fixture here exercises it. Left as a documented contract stub for T068's own implementation tests.
- **VC-004's purge-on-`FLAKY` / `INPUTS_UNSOUND`** rule is a cross-tool integration with `gates/flake_ledger.py` (T-C09) and `gates/io_trace.sh` (T-C02) state this task does not own; `test_flake_ledger_red.sh` (T059) is the RED test that contract-binds "never cached" for a flaky gate.
- **VC-005's periodic backstop lane** is T-C10's own scope (`test_backstop_red.sh`, T060).